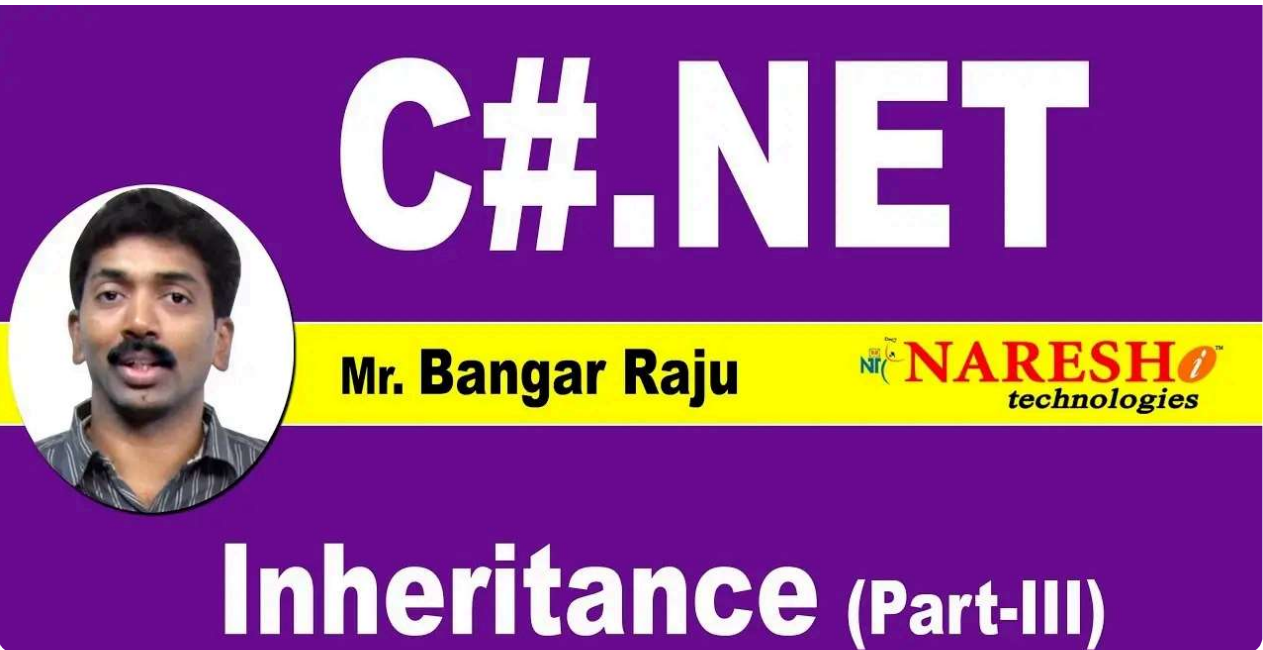




# Understanding Inheritance in C#.NET: A Comprehensive Guide

By Albert Harmon   Published August 26, 2016   ⌚ 3 min read   [Education](#)



**TLDR** This blog post explores the concept of inheritance in C#.NET, detailing its importance in application development, the identification of entities and attributes, and the hierarchical organization of classes to

# eliminate redundancy and promote reusability.

Inheritance is a fundamental concept in object-oriented programming that allows developers to create a new class based on an existing class. In this blog post, we will delve into how inheritance can be effectively utilized in C#.NET applications, focusing on its implementation, rules, and practical examples.

Inheritance is not just a feature that can be added in the middle of a project; it is often planned during the initial stages of application development. By understanding how to implement inheritance, developers can create more efficient and organized code.

## What is an Entity?

In the context of database management systems (DBMS), an entity is defined as a living or non-living object associated with a set of attributes. For instance, in a banking application, a customer is an entity, while in a school application, a student serves as an entity. Each entity is characterized by specific attributes:

- **Customer:** ID, name, address, phone number
- **Student:** ID, name, class, marks
- **Employee:** ID, name, job title, salary

## Examples of Entities

Entities can be both living and non-living. For example:

- A **phone** can be considered an entity with attributes like manufacturer, model number, price, and features.
- A **pen** can also be an entity with attributes such as color, height, and diameter.

## Steps to Implement Inheritance

When developing an application, the following steps should be followed to effectively implement inheritance:

### Step 1: Identify Entities

The first step is to identify the entities associated with the application. For example, in a school application, the entities might include:

- Student
- Teaching Staff
- Non-Teaching Staff

### Step 2: Identify Attributes

Next, identify the attributes for each entity. For instance:

- **Student:** ID, name, address, phone number, class, marks, fees

- **Teaching Staff:** ID, name, address, phone number, designation, salary, qualification, subject
- **Non-Teaching Staff:** ID, name, address, phone number, designation, salary, department

## Step 3: Identify Common Attributes

After identifying the attributes, the next step is to find common attributes among the entities. Common attributes might include:

- ID
- Name
- Address
- Phone Number

## Step 4: Define Classes Hierarchically

To avoid code duplication, define a parent class that contains the common attributes. For example, a `Person` class can be created to hold the common attributes:

```
public class Person {  
    public int ID;  
    public string Name;  
    public string Address;
```

```
    public string Phone;
}
```

Then, create child classes for each specific entity:

```
public class Student : Person {
    public float Marks;
    public string Grade;
    public double Fees;
}
```

```
public class Staff : Person {
    public string Designation;
    public double Salary;
}
```

```
public class TeachingStaff : Staff {
    public string Qualification;
    public string Subject;
}
```

```
public class NonTeachingStaff : Staff {
    public string Department;
    public int ManagerID;
}
```

# Benefits of Using Inheritance

The primary advantage of using inheritance is reusability. By defining common attributes in a parent class, developers can avoid redundancy and streamline their code. This not only makes the code cleaner but also easier to maintain and extend in the future.

## Conclusion

Inheritance is a powerful feature in C#.NET that enhances the organization and efficiency of application development. By following the steps outlined in this guide, developers can effectively implement inheritance, ensuring their applications are robust and maintainable. Understanding and applying these principles from the outset of a project can lead to significant improvements in code quality and development speed.